Modern WPF Practice Project — Design

Goal: A single, end-to-end practice project that exercises nearly all important WPF / MVVM concepts you've learned: DI, Host, appsettings, logging, structured logging (Serilog), multiple windows, modal & non-modal dialogs, file dialogs, data binding, validation, commands, behaviors, styles & themes, custom controls, attached properties, templates, virtualization, asynchronous patterns, message bus, unit testing, and a small persisted data store (SQLite + EF Core).

This document describes the app concept, user flows, windows, architecture, folder layout, required packages, implementation notes, and suggested milestones.

Project concept — "TaskFlow Studio"

A lightweight **task/project manager** for individual developers that demonstrates real desktop concerns:

- Create/organize tasks into projects
- Track task status, priority, tags, due dates
- Import / export tasks (CSV/JSON)
- Simple local persistence (SQLite)
- Non-modal little tool window that emits live progress / status
- Modal dialogs for input/edit and confirmation
- Settings window with theming and app preferences
- Background autosave / sync simulation (background service)
- Search + filtering + sorting + grouping + virtualization for large lists

Why this is good practice: it has UI complexity (lists, forms), multiple windows, validation, long-running operations, local persistence, and places where MVVM patterns shine.

Core user stories (high level)

1. As a user I can create a new project and add tasks to it.
2. As a user I can open a modal dialog to create/edit a task and the dialog returns the task to Main.
3. As a user I can open a non-modal "Live Monitor" window that publishes progress updates (simulate working on a task) and Main updates in real-time.
4. As a user I can import/export tasks using File Open/Save dialogs.
5. As a user I can change app theme and preferences in a Settings window; settings persist in `appsettings.json` (or user settings store).
6. As a user I can search, filter, group and sort tasks. UI responses stay snappy with virtualization for many tasks.
7. As a developer I can run unit tests for ViewModels and services (mocking dialog/file services).

Window list & responsibilities

MainWindow

- Shows list of projects (left) and the selected project's tasks (right).
- Toolbar: New Project, New Task (modal), Import, Export, Open Live Monitor.
- Search box and filters (status, tag, priority). Supports grouping and sorting.
- Uses `IWindowService` and `IDialogService` for windows and dialogs.

TaskEditWindow (modal)

- Modal dialog bound to `TaskEditViewModel`.
- Fields: Title (required), Description, DueDate, Priority, Status, Tags.
- OK/Cancel. Uses validation (INotifyDataErrorInfo) to enable OK only when valid.
- Returns the created/edited Task via `IDialogService.ShowDialog(vm)` and `vm.DialogResult`.

LiveMonitorWindow (non-modal)

- Non-modal tool window for live updates.
- Publishes `LiveUpdate` messages via `IMessageBus`. Can start/stop simulated work.
- MainWindow subscribes to `LiveUpdate` events to show progress to the user.

SettingsWindow (modal or non-modal)

- Edit app preferences: theme (Light/Dark), autosave frequency, window size.
- Uses `IOptions<T>` to retrieve settings and a small `ISettingsService` to persist user settings.

AboutWindow (non-modal)

- Static info, license, version. Use `Assembly` metadata to show version.

Architecture & Patterns

- **MVVM**: ViewModels have no WPF types; they use abstractions for dialogs and windows.
- **DI**: Use `Host.CreateDefaultBuilder()` in `App.xaml.cs` to create `IHost` and register services.
- **Logging**: Serilog via host; logs to rolling files + console + optional Seq.
- **Configuration**: `appsettings.json` + environment overrides. Bind to POCOs using `IOptions<T>`.
- **Persistence**: EF Core with SQLite (local file) for tasks and projects.
- **Messaging**: `IMessageBus` for decoupled non-modal updates.
- **Dialog & Window services**: `IDialogService` for modal dialogs; `IWindowService` for showing windows.
- **Validation**: `INotifyDataErrorInfo` in dialog ViewModels.

- **Async**: Use `async/await` and `Task.Run` only for CPU-bound work; `ConfigureAwait(false)` in libraries; cancellation tokens for long tasks.
- **Unit testing**: ViewModels tested with fake `IDialogService` and `IMessageBus`.

Suggested packages

- Microsoft.Extensions.Hosting
- Microsoft.Extensions.DependencyInjection
- Microsoft.Extensions.Configuration.Json
- Microsoft.Extensions.Options
- Serilog.AspNetCore (or Serilog.Extensions.Hosting) + Serilog.Sinks.File + Serilog.Sinks.Console
- Microsoft.EntityFrameworkCore.Sqlite
- Microsoft.EntityFrameworkCore.Design
- CommunityToolkit.Mvvm (optional) or your existing RelayCommand / ViewModelBase
- NUnit/xUnit + Moq for tests

Folder / Project layout

```
/TaskFlowStudio
  /Models
    Project.cs
    TaskItem.cs
  /ViewModels
    MainViewModel.cs
    ProjectViewModel.cs
    TaskEditViewModel.cs
    LiveMonitorViewModel.cs
    SettingsViewModel.cs
  /Views
    MainWindow.xaml
    TaskEditWindow.xaml
    LiveMonitorWindow.xaml
    SettingsWindow.xaml
  /Services
    IDialogService.cs
    DialogService.cs
    IWindowService.cs
    WindowService.cs
    IMessageBus.cs
    MessageBus.cs
    ISettingsService.cs
    SettingsService.cs
    IRepository.cs
    EfRepository.cs
```

```
  /Infrastructure
    RelayCommand.cs
    ViewModelBase.cs
    NotificationHelpers.cs
  /Data
    AppDbContext.cs
    Migrations/...
  /Resources
    Themes/Light.xaml
    Themes/Dark.xaml
    Styles/Controls.xaml
  appsettings.json
  App.xaml
  App.xaml.cs
  TaskFlowStudio.csproj
/tests/TaskFlowStudio.Tests
```

Key implementation points (copy-paste snippets)

> Note: full code will be long — below are the most important patterns and snippets you should implement.

Host & App startup (App.xaml.cs)

```
public partial class App : Application
{
    private IHost? _host;

    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);

        _host = Host.CreateDefaultBuilder()
            .ConfigureAppConfiguration((ctx, cb) =>
            {
                cb.SetBasePath(AppContext.BaseDirectory)
                  .AddJsonFile("appsettings.json", optional: false,
reloadOnChange: true)
                  .AddEnvironmentVariables();
            })
            .ConfigureServices((ctx, services) =>
            {
                // config
```

```
services.Configure<ApplicationSettings>(ctx.Configuration.GetSection("Application"));

                // db
                services.AddDbContext<AppDbContext>(options =>
                    options.UseSqlite($"Data
Source={Path.Combine(AppContext.BaseDirectory, "taskflow.db")}"));

                // services
                services.AddSingleton<IMessageBus, MessageBus>();
                services.AddSingleton<IDialogService, DialogService>();
                services.AddSingleton<IWindowService, WindowService>();
                services.AddSingleton<IDictionary<Type, Type>>(sp => new
Dictionary<Type, Type>
                {
                    { typeof(MainViewModel), typeof(MainWindow) },
                    { typeof(TaskEditViewModel), typeof(TaskEditWindow) },
                    { typeof(LiveMonitorViewModel), typeof(LiveMonitorWindow) },
                    { typeof(SettingsViewModel), typeof(SettingsWindow) }
                });

                // viewmodels
                services.AddTransient<MainViewModel>();
                services.AddTransient<TaskEditViewModel>();
                services.AddTransient<LiveMonitorViewModel>();
                services.AddTransient<SettingsViewModel>();

                // repositories
                services.AddScoped<IRepository<Project>,
EfRepository<Project>>();
                services.AddScoped<IRepository<TaskItem>,
EfRepository<TaskItem>>();

                // hosted background service
                services.AddHostedService<AutoSaveService>();
            })
            .UseSerilog((ctx, cfg) =>
            {
                cfg.ReadFrom.Configuration(ctx.Configuration);
                cfg.WriteTo.File("Logs/log-.txt", rollingInterval:
RollingInterval.Day);
                cfg.Enrich.FromLogContext();
            })
            .Build();

        _host.Start();

        var windowService = _host.Services.GetRequiredService<IWindowService>();
        windowService.Show<MainViewModel>();
```

```
    }

    protected override async void OnExit(ExitEventArgs e)
    {
        if (_host is not null)
        {
            await _host.StopAsync();
            _host.Dispose();
        }

        base.OnExit(e);
    }
}
```

IDialogService — modal dialog pattern

```
public interface IDialogService
{
    bool ShowDialog<TViewModel>(TViewModel viewModel) where TViewModel : class;
}

public class DialogService : IDialogService
{
    private readonly IServiceProvider _provider;
    private readonly IDictionary<Type, Type> _map;

    public DialogService(IServiceProvider provider, IDictionary<Type, Type> map)
    {
        _provider = provider;
        _map = map;
    }

    public bool ShowDialog<TViewModel>(TViewModel viewModel) where TViewModel :
class
    {
        var windowType = _map[typeof(TViewModel)];
        var window = (Window)Activator.CreateInstance(windowType)!;
        window.DataContext = viewModel;

        if (viewModel is ICloseRequestNotifier notifier)
        {
            notifier.RequestClose += (r) => { window.DialogResult = r;
window.Close(); };
        }

        return window.ShowDialog() == true;
```

```
    }
}
```

IMessageBus — tiny pub/sub

```
public interface IMessageBus
{
    void Publish<T>(T msg);
    void Subscribe<T>(Action<T> handler);
    void Unsubscribe<T>(Action<T> handler);
}
```

(Use same `MessageBus` from earlier design; thread-safe simple implementation.)

TaskEditViewModel — validation using INotifyDataErrorInfo

- Implement `INotifyDataErrorInfo` to provide errors for Title required and DueDate in past.
- `OkCommand` only enabled when `!HasErrors`.

EF Core model sample

```
public class TaskItem
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string? Description { get; set; }
    public DateTime? DueDate { get; set; }
    public string Status { get; set; } = "Todo";
    public int ProjectId { get; set; }
}
```

UX / UI details

- Use MVVM-friendly `DataTemplate`s for list items; `ObservableCollection<T>` for collections.
- Use `CollectionViewSource` for filtering / sorting / grouping (expose `ICollectionView` from VM).
- Use `VirtualizingStackPanel` and `EnableCollectionVirtualization` for large lists.
- Provide keyboard shortcuts and accessibility names.
- Use resource dictionaries for colors, spacing, and fonts. Add `Light.xaml` and `Dark.xaml` and make theme switching via merged dictionaries.
- Prefer `Button` command bindings and `CommandParameter` for context actions.

Testing strategy

- Unit test `MainViewModel` and `TaskEditViewModel` using fake `IDialogService`, fake `IRepository`, and fake `IMessageBus`.
- Integration tests: In-memory EF Core provider for repository tests.
- UI tests (optional): Playwright for desktop or WinAppDriver for automated UI flows.

Milestones (recommended)

1. Project skeleton + DI + Host + appsettings + logging
2. Models + EF Core and migrations, repository pattern
3. MainWindow with list and basic create/edit via simple forms (no modal) to exercise data binding
4. Implement IDialogService + TaskEditWindow modal flow
5. Add IMessageBus + LiveMonitor non-modal updates
6. Add validation (INotifyDataErrorInfo) and unit tests
7. Add import/export using FileDialogService and settings window
8. Add virtualization, grouping, search, and performance improvements
9. Polish styles/themes, add icons and resources
10. Add CI for tests and build

Extras (stretch tasks)

- Add undo/redo for task edits (command history)
- Add plugin architecture for custom exporters (MEF or simple plugin loader)
- Add telemetry integration and feature flags
- Add multi-window docking using AvalonDock

Final notes

This practice project intentionally mixes many real-world pieces: Host, DI, appsettings, logging, EF Core, multiple windows, modal & non-modal flows, and testability. It’s large enough to be a portfolio piece while still approachable to build incrementally.

If you want, I can now:

- ✨Generate a **runnable Visual Studio starter template** (complete csproj + files) for the project skeleton (Host + DI + MainWindow + TaskEdit modal dialog + services).
- 🦝Or, generate **one complete feature at a time** (start with Host + DI + appsettings + a skeleton MainWindow and window service).

Which would you like me to create next?